SMART CONTRACT AUDIT REPORT

for

Atomica Protocol

Prepared By: Xiaomi Huang

PeckShield

March 30, 2026

Document Properties

Client	Atomica
Title	Smart Contract Audit Report
Target	Atomica
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang, Xiaomi Huang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	March 30, 2026	Xuxian Jiang	Final Release
1.0-rc1	March 3, 2026	Xuxian Jiang	Release Candidate #1
0.8	March 1, 2026	Xuxian Jiang	Initial Version

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction 4
- 1.1 About Atomica . . . 4
- 1.2 About PeckShield . . . 5
- 1.3 Methodology . . . 5
- 1.4 Disclaimer . . . 6

2 Findings 9
- 2.1 Summary . . . 9
- 2.2 Key Findings . . . 10

3 Detailed Results 11
- 3.1 Improper Inflow Stream Deactivation Logic in deactivateInflowStream() . . . 11
- 3.2 Revisited maxTotalEmergencyPauseDurationPerMonth Enforcement . . . 13
- 3.3 Possibly Zero Interest Repayment Over Fixed Financing . . . 14
- 3.4 Improper Principal Repayment Logic in FinancingMarketAdapter . . . 16
- 3.5 Timely Inflow Stream Update Upon Pool Share Mints And Burns . . . 18
- 3.6 Improved Parameter Validation in FundLib::_tryBorrowFromIntent() . . . 19
- 3.7 Inaccurate Borrow Rate Calculation Logic in FundLib . . . 21
- 3.8 Improved Precision in Inflow Stream Rate Calculation . . . 24
- 3.9 Improper Weighted Rate Accounting During Write-Off . . . 26
- 3.10 Revisited fixedInterestObligations Accounting in payInterest() . . . 28
- 3.11 Accommodation of Non-ERC20-Compliant Tokens . . . 30
- 3.12 Trust Issue of Admin Keys . . . 32

4 Conclusion 35

References 36

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the core Atomica protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About Atomica

Atomica is an algorithmic protocol that aims to provide financial flexibility and reduce risk for protocol users by offering a range of protection cover options and underwriting capital for liquidity providers. This audit covers the core financing markets to enable peer-to-peer financing on top of pools, with intent-based matching, access control (via ERC721 tokens), and an extensible hooks system. Each pool is in essence an ERC4626 vault with integrated access control, inflow streams, phased lifecycle, and outflow processors. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The Atomica Protocol

Item	Description
Client	Atomica
Type	EVM Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	March 30, 2026

In the following, we show the Git repositories of reviewed files and the commit hash values used in this audit.

- https://github.com/Atomica-Core/Lending.git (76945cb)

- https://github.com/Atomica-Core/Pools.git (81b6493)

And here are the commit IDs after all fixes for the issues found in the audit have been checked in:

- https://github.com/Atomica-Core/Financing-Markets.git (aa3b7c9)
- https://github.com/Atomica-Core/Pools.git (4dad524)

1.2 About PeckShield

PeckShield Inc. [11] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification

Impact \ Likelihood	*High*	*Medium*	*Low*
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [9], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 | Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the core `Atomica` protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	1	■
Medium	5	■ ■ ■ ■ ■
Low	6	■ ■ ■ ■ ■ ■
Informational	0	
Total	12	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 high-severity vulnerability, 5 medium-severity vulnerabilities, and 6 low-severity vulnerabilities.

Table 2.1: Key Atomica Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Improper Inflow Stream Deactivation Logic in deactivateInflowStream()	Coding Practices	Resolved
PVE-002	Low	Revisited maxTotalEmergencyPauseDurationPerMonth Enforcement	Business Logic	Resolved
PVE-003	High	Possibly Zero Interest Repayment Over Fixed Financing	Business Logic	Resolved
PVE-004	Medium	Improper Principal Repayment Logic in FinancingMarketAdapter	Business Logic	Resolved
PVE-005	Medium	Timely Inflow Stream Update Upon Pool Share Mints And Burns	Business Logic	Resolved
PVE-006	Low	Improved Parameter Validation in FundLib::_tryBorrowFromIntent()	Coding Practices	Resolved
PVE-007	Low	Inaccurate Borrow Rate Calculation Logic in FundLib	Business Logic	Resolved
PVE-008	Low	Improved Precision in Inflow Stream Rate Calculation	Numeric Errors	Resolved
PVE-009	Medium	Improper Weighted Rate Accounting During Write-Off	Business Logic	Resolved
PVE-010	Medium	Revisited fixedInterestObligations Accounting in payInterest()	Business Logic	Resolved
PVE-011	Low	Accommodation of Non-ERC20-Compliant Tokens	Coding Practices	Resolved
PVE-012	Medium	Trust on Admin Keys	Security Features	Mitigated

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Improper Inflow Stream Deactivation Logic in deactivateInflowStream()

- ID: PVE-001
- Severity: Low
- Likelihood: Medium
- Impact: Low
- Target: `InflowStreamImpl`
- Category: Coding Practices [6]
- CWE subcategory: CWE-1126 [1]

Description

The `Atomica` protocol allows for multiple time-based token distribution streams for pool share holders. The support is mainly implemented in a core `InflowStreamImpl` contract that manages the full lifecycle of inflow streams from creation to claiming/archiving. While examining the logic to deactivate a stream (after the associated claim window) expires, we notice an issue that needs to be addressed.

In the following, we show the implementation of the related function, i.e., `deactivateInflowStream()`. It has a rather straightforward logic in validating the caller and the given input arguments, next locating the inflow stream, and then removing it from active distribution. It comes to our attention that it has an assertion, i.e., `assert(index < len - 1)` (line 576), to locate the inflow stream for deactivation. This assertion is not needed. Note the same issue is also applied to another routine `detachInflowDropToken()` where a drop token is removed from tracking.

```
540     function deactivateInflowStream(
541         uint256 inflowStreamId_
542     ) external override nonReentrant {
543         if (inflowStreamId_ == 0 || inflowStreamId_ > inflowStreamIdInc) {
544             revert ErrInvalidInflowStreamId();
545         }

547         InflowStream storage inflowStream = inflowStreams[inflowStreamId_];

549         // Only from creator, keeper or accounts from maintainersList (if it defined)
```

```
        if (
            msg.sender != inflowStream.creator &&
            msg.sender != keeper &&
            (maintainersListId == 0 ||
                !list.passList(maintainersListId, msg.sender))
        ) {
            revert ErrNotAllowed();
        }

        if (inflowStream.deactivated) {
            revert ErrInflowStreamDeactivated();
        }

        if (block.timestamp < inflowStream.endTime + inflowStreamClaimWindow) {
            revert ErrInflowStreamNotFinished();
        }

        inflowStream.deactivated = true;

        uint256 index = 0;
        uint256 len = activeInflowStreams.length;
        for (; index < len; index++) {
            if (activeInflowStreams[index] == inflowStreamId_) {
                break;
            }

            assert(index < len - 1);
        }

        activeInflowStreams[index] = activeInflowStreams[len - 1];
        activeInflowStreams.pop();

        emit LogInflowStreamDeactivated(inflowStreamId_);
    }
```

Listing 3.1: `InflowStreamImpl::deactivateInflowStream()`

Recommendation Improve the above-mentioned routines to remove unneeded assertions.

Status This issue has been fixed by this commit: `6b09c63`.

3.2 Revisited maxTotalEmergencyPauseDurationPerMonth Enforcement

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `PoolFactory`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The `Atomica` protocol has a `PoolFactory` contract for deploying and managing pool instances with emergency pause functionality. As part of design, it enforces `maxTotalEmergencyPauseDurationPerMonth`, a risk parameter that enforces an immutable maximum pause duration per `30`-day period. While reviewing the logic to enforce this specific risk parameter, we notice it does not strictly achieve the intended enforcement.

To elaborate, we show below the implementation of the related function, i.e., `pause()`, that implements emergency pause with monthly-duration tracking. We notice the internal track of pause duration per month is reset when the following condition is met, i.e., `block.timestamp > emergencyPause.firstTimeStartedAt + MONTH` (line 198). However, this condition may not be appropriate as the reset should be applied when the last update, not first time update, is more than one month ago. In fact, a precise enforcement requires to maintain an array of pause-unpause timestamps, i.e., `[pauseAt, unpauseAt]`, filter this array in the last month, and then add all resulting seconds for the pause duration calculation.

```
186     function pause() external override {
187         if (!IList(list).passList(emergencyPauseListId, msg.sender)) {
188             revert ErrNotAllowed();
189         }
190 
191         if (paused) {
192             revert ErrNotAllowed();
193         }
194 
195         // Not allowed to pause if total pause duration exceeds
                maxTotalEmergencyPauseDurationPerMonth
196         if (
197             emergencyPause.firstTimeStartedAt == 0 ||
198             block.timestamp > emergencyPause.firstTimeStartedAt + MONTH
199         ) {
200             if (maxTotalEmergencyPauseDurationPerMonth == 0) {
201                 revert ErrNotAllowed();
202             }
203 
```

```
            emergencyPause = EmergencyPause(
                block.timestamp,
                block.timestamp,
                0
            );
        } else {
            if (
                emergencyPause.totalDuration >=
                maxTotalEmergencyPauseDurationPerMonth
            ) {
                revert ErrMaxEmergencyPauseDurationExceeded(
                    emergencyPause.totalDuration
                );
            }

            emergencyPause.lastTimeStartedAt = block.timestamp;
        }

        paused = true;

        emit Paused(msg.sender);
    }
```

Listing 3.2: `PoolFactory::pause()`

Recommendation Improve the above routine to ensure the intended risk parameter is properly enforced, i.e., `maxTotalEmergencyPauseDurationPerMonth`.

Status This issue has been fixed by this commit: `5f4cd2d`.

3.3 Possibly Zero Interest Repayment Over Fixed Financing

- ID: PVE-003
- Severity: High
- Likelihood: High
- Impact: High
- Target: `FinancingMarketAdapter`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

To actually fund a financing request, `Atomica` allows for asset borrowing from a so-called adapter that bridges lending pools and outflows for liquidity management. Currently, there are two types of financing for interest calculation, i.e., `INTEREST_ACCRUAL` and `FIXED_REPAYMENT`. The first type accrues borrow interest depends on actual duration while the second type has a fixed upfront repayment amount at financing disbursement and the interest does not change regardless of when borrower

repays. We notice the funding of the second type has an issue that allows for nearly zero interest repayment.

To elaborate, we show below the code snippet of the related routine, i.e., `fund()`, that allows to borrow assets from this adapter using a specific intent. When the financing type is `FIXED_REPAYMENT`, it simply sets the obligation interest rate to zero (`obligationsById[id_].rate = 0`) without validating the fixed interest over the loan duration matches the user-specified rate in the intent. As a result, current implementation allows to fund a borrow without paying meaningful interest, a violation of the intended user intent design.

```
    function fund(
        uint96 intentId_,
        uint256 amount_,
        address recipient_,
        FinancingType financingType_,
        uint256 fixedInterest_
    )
        external
        override
        nonReentrant
        intentExists(intentId_)
        returns (uint256 id_, uint256 borrowedAmount_)
    {
        ...
        obligationsById[id_].borrowedPrincipal = borrowedAmount_;
        obligationsById[id_].lastUpdateTs = block.timestamp;
        obligationsById[id_].market = msg.sender;
        obligationsById[id_].intentId = intentId_;
        obligationsById[id_].financingType = financingType_;

        if (lastUpdateTs > 0) {
            accruedInterest += weightedRate * (block.timestamp - lastUpdateTs);
        }

        totalBorrowedPrincipal += borrowedAmount_;
        lastUpdateTs = block.timestamp;

        if (financingType_ == FinancingType.FIXED_REPAYMENT) {
            // Fixed: rate=0, store fixed interest pre-scaled
            obligationsById[id_].rate = 0;
            obligationsById[id_].accruedInterest = fixedInterest_ * _ONE;
            accruedInterest += fixedInterest_ * _ONE; // Immediately in totalAssets()
            fixedInterestObligations += fixedInterest_;
            // weightedRate unchanged (rate is 0)
        } else {
            // Accrual: use intent rate, time-based accrual
            obligationsById[id_].rate = intent.rate;
            weightedRate += borrowedAmount_ * intent.rate;
        }

        // Update intent's funded amount
```

```
        intent.amountFilled += borrowedAmount_;

        CallHelper.safeTransfer(asset, recipient_, borrowedAmount_);

        emit LogFunded(id_, borrowedAmount_, obligationsById[id_].rate, intentId_);
    }
```

Listing 3.3: `FinancingMarketAdapter::fund()`

Recommendation Revise the above `fund()` routine to properly ensure both types meet the intended interest rate specified in the user intent.

Status This issue has been fixed by this commit: `12370cb`.

3.4 Improper Principal Repayment Logic in FinancingMarketAdapter

- ID: PVE-004
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: `RepaymentLib`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The `Atomica` protocol has a core utility `RepaymentLib` contract for repayment-related logic. In the process of reviewing the principal-related repayment, we notice an issue that may improperly mark a financing request is fully repaid when there is still pending interest.

In the following, we show the implementation of the related function, i.e., `repayPrincipal()`. It has a rather straightforward logic in repaying principal across financing tranches sequentially. Note that it is only paying the principal for all financing tranches. In other words, it makes zero interest payment. However, it reports back its caller the financing as fully repaid at the end when there is no pending principal regardless of pending interest. To fix, we need to ensure the financing has zero pending principal and zero pending interest before making it as fully repaid.

```
    function repayPrincipal(
        IFinancingMarket.Tranche[] storage tranches_,
        address asset_,
        uint256 amount_,
        uint256 financingId_
    ) external returns (uint256 repaid_, bool fullyRepaid_) {
        uint256 remainder = amount_;

        for (uint256 i = 0; i < tranches_.length; i++) {
```

```
            if (remainder == 0) break;

            IFinancingMarket.Tranche storage tranche = tranches_[i];
            IFinancingMarketAdapter adapter = IFinancingMarketAdapter(
                    tranche.adapter
                );

            uint256 obligation = adapter.principalObligations(tranche.obligationId);
            if (obligation == 0) continue;

            uint256 repayAmount = MathHelper.min(obligation, remainder);

            CallHelper.safeApprove(asset_, address(adapter), repayAmount);

            uint256 repaidAmount = adapter.repayPrincipal(
                tranche.obligationId,
                repayAmount
            );

            remainder -= repaidAmount;
            repaid_ += repaidAmount;

            emit IFinancingMarket.LogPrincipalRepaid(
                financingId_,
                tranche.pool,
                repaidAmount,
                i
            );
        }

        fullyRepaid_ = _calculatePrincipalObligations(tranches_) == 0;
    }
```

Listing 3.4: `RepaymentLib::repayPrincipal()`

Recommendation Improve the above routine to ensure a financing is indeed fully repaid when both the principal and interest are paid off.

Status This issue has been fixed by this commit: `b17fab6`.

3.5 Timely Inflow Stream Update Upon Pool Share Mints And Burns

- ID: PVE-005
- Severity: Medium
- Likelihood: Medium
- Impact: Medium
- Target: `PoolImpl`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The pool in `Atomica` is an `ERC4626`-style vault pool with advanced features and proxy pattern. In particular, it allows to stream inflow tokens to share holders proportionally. While reviewing the `ERC20`-associated functionality in the vault pool, we notice the implication on timely updating inflow streams. And current implementation is incomplete especially when pool tokens are minted and burned.

To elaborate, we show below the implementation of the underlying `_move()` function that moves pool tokens between addresses. While it properly calls `updateAccount()` before balance changes for both sender and recipient, it forgets the need to also timely call `updateAccount(address(0))` before the `totalSupply` state is updated.

```
    function _move(
        address sender_,
        address recipient_,
        uint256 shares_
    ) internal {
        if (sender_ == address(0)) {
            totalSupply_ += shares_;
        } else {
            updateAccount(sender_);

            uint256 senderBalance = balances[sender_];

            if (senderBalance < shares_) {
                revert ErrInsufficientBalance(sender_, senderBalance, shares_);
            }

            balances[sender_] -= shares_;
        }

        if (recipient_ == address(0)) {
            totalSupply_ -= shares_;
        } else {
            updateAccount(recipient_);
            balances[recipient_] += shares_;
        }
```

```

        emit Transfer(sender_, recipient_, shares_);
    }
```

Listing 3.5: `PoolImpl::_move()`

Recommendation Revisit the above routine to ensure rewards are calculated based on balances before the transfer. It is necessary for accurate inflow stream and drop accounting.

Status This issue has been fixed by this commit: `ac2e686`.

3.6 Improved Parameter Validation in FundLib::_tryBorrowFromIntent()

- ID: PVE-006
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `Rentable`
- Category: Coding Practices [6]
- CWE subcategory: CWE-1126 [1]

Description

`DeFi` protocols typically have a number of system-wide parameters that can be dynamically configured on demand. The `Atomica` protocol is no exception. Specifically, if we examine the lending intent created by a liquidity provider offering funds at specific terms, the `intent` structure contains `validFrom` and `deadlineTs` to specify the time period where the intend is valid. In the following, we show a specific routine that can be improved to enforce both `validFrom` and `deadlineTs`, not only `deadlineTs`.

```
    function _tryBorrowFromIntent(
        IFinancingMarket.Tranche[] storage tranches_,
        FundParams memory params_,
        RequestData memory request_,
        bytes32 encodedIntent_,
        uint256 maxAmount_
    ) internal returns (uint256 borrowed_, uint256 rate_) {
        (address adapter, uint96 intentId) = IntentIdHelper.decodeIntentId(
            encodedIntent_
        );

        // Validate adapter is from FinancingMarketFactory
        if (!IFinancingMarketFactory(params_.factory).adapters(adapter)) {
            return (0, 0);
        }

        // Validate adapter is allowed (if list is configured)
```

```
        if (params_.allowedAdaptersListId != 0) {
            if (
                !IList(params_.list).passList(
                    params_.allowedAdaptersListId,
                    adapter
                )
            ) {
                return (0, 0);
            }
        }

        // Get intent details and validate
        IFinancingMarketAdapter.Intent
            memory intent = IFinancingMarketAdapter(adapter).intents(
                intentId
            );

        if (intent.market != address(this)) {
            return (0, 0);
        }

        if (intent.deadlineTs > 0 && intent.deadlineTs <= block.timestamp) {
            return (0, 0);
        }

        // Check intent filters
        if (!_passIntentFilters(params_.list, intent.intentFilter, request_)) {
            return (0, 0);
        }

        // Calculate amount to fund
        uint256 amountToBorrow = MathHelper.min(
            intent.amount - intent.amountFilled,
            maxAmount_
        );

        if (amountToBorrow == 0) {
            return (0, 0);
        }

        rate_ = intent.rate;

        // Borrow from adapter and create tranche
        {
            (
                uint256 obligationId,
                uint256 actualBorrowed
            ) = _borrowFromAdapter(adapter, intentId, amountToBorrow, request_);

            if (actualBorrowed == 0) {
                return (0, 0);
            }
```

```

            borrowed_ = actualBorrowed;

            _createTranche(
                tranches_,
                params_.financingId,
                adapter,
                obligationId,
                actualBorrowed,
                rate_
            );
        }
    }
```

Listing 3.6: FundLib::_tryBorrowFromIntent()

We need to point out that the `validFrom` parameter is actually enforced inside the called `_borrowFromAdapter()` function, which, though functionally correct, does introduce possible inconsistency in intent validation. Meanwhile, the lending intent for `FIXED_REPAYMENT` (Section 3.3) should be enforced for having non-zero duration inside `FinancingMarket::_requestFinancing()`. All in all, these parameters define various aspects of the protocol operation and maintenance and need to exercise extra care when configuring, updating, or enforcing them.

Recommendation Validate any changes regarding these risk parameters to ensure they fall in an appropriate range.

Status This issue has been fixed by this commit: `2c4b10c`.

3.7 Inaccurate Borrow Rate Calculation Logic in FundLib

- ID: PVE-007
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `FundLib`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

As mentioned in Section 3.6, a lending user may create an intent to offer funds at specific terms. The term includes the fund amount, interest rate, and the valid time duration. Each intent also tracks how much has been borrowed and enforces filters on who can fund. While examining the logic to fund a specific financing request a user intent, we notice current approach to compute the borrow rate may be improved.

In the following, we show the implementation of the related function, i.e., `_tryBorrowFromIntent()`. By design, it validates the given adapter and intent, and then borrows and creates a financing tranche. As mentioned in Section 3.3, there are two types of financing for interest calculation, i.e., `INTEREST_ACCRUAL` and `FIXED_REPAYMENT`. The first type accrues borrow interest depends on actual duration while the second type has a fixed upfront repayment amount. We notice the second type should compute the borrow rate differently. Instead of current approach of directly using the intent rate, we shall compute it for the `FIXED_REPAYMENT` case as follows: `(request_.repayAmount - request_.amount)* _ONE / (request_.amount * request_.duration)`.

```
    function _tryBorrowFromIntent(
        IFinancingMarket.Tranche[] storage tranches_,
        FundParams memory params_,
        RequestData memory request_,
        bytes32 encodedIntent_,
        uint256 maxAmount_
    ) internal returns (uint256 borrowed_, uint256 rate_) {
        (address adapter, uint96 intentId) = IntentIdHelper.decodeIntentId(
            encodedIntent_
        );

        // Validate adapter is from FinancingMarketFactory
        if (!IFinancingMarketFactory(params_.factory).adapters(adapter)) {
            return (0, 0);
        }

        // Validate adapter is allowed (if list is configured)
        if (params_.allowedAdaptersListId != 0) {
            if (
                !IList(params_.list).passList(
                    params_.allowedAdaptersListId,
                    adapter
                )
            ) {
                return (0, 0);
            }
        }

        // Get intent details and validate
        IFinancingMarketAdapter.Intent
            memory intent = IFinancingMarketAdapter(adapter).intents(
                intentId
            );

        if (intent.market != address(this)) {
            return (0, 0);
        }

        if (intent.deadlineTs > 0 && intent.deadlineTs <= block.timestamp) {
            return (0, 0);
        }
```

```
        if (block.timestamp < intent.validFrom) {
            return (0, 0);
        }

        // Check intent filters
        if (!_passIntentFilters(params_.list, intent.intentFilter, request_)) {
            return (0, 0);
        }

        // Calculate amount to fund
        uint256 amountToBorrow = MathHelper.min(
            intent.amount - intent.amountFilled,
            maxAmount_
        );

        if (amountToBorrow == 0) {
            return (0, 0);
        }

        rate_ = intent.rate;

        // For fixed repayment: validate borrower's fixed interest meets LP's minimum
            rate
        if (
            request_.financingType ==
            IFinancingMarketAdapter.FinancingType.FIXED_REPAYMENT &&
            request_.duration > 0 &&
            (request_.repayAmount - request_.amount) * _ONE <
                intent.rate * request_.amount * request_.duration
        ) {
            return (0, 0);
        }

        // Borrow from adapter and create tranche
        {
            (
                uint256 obligationId,
                uint256 actualBorrowed
            ) = _borrowFromAdapter(adapter, intentId, amountToBorrow, request_);

            if (actualBorrowed == 0) {
                return (0, 0);
            }

            borrowed_ = actualBorrowed;

            _createTranche(
                tranches_,
                params_.financingId,
                adapter,
                obligationId,
```

```
                actualBorrowed,
                rate_
            );
        }
    }
```

Listing 3.7: `FundLib::_tryBorrowFromIntent()`

Recommendation Revise the above routine to compute the intended borrow rate.

Status This issue has been fixed by this commit: `6f9b8c9`.

3.8 Improved Precision in Inflow Stream Rate Calculation

- ID: PVE-008
- Severity: Low
- Likelihood: Medium
- Impact: Low
- Target: `Multiple Contracts`
- Category: Numeric Errors [8]
- CWE subcategory: CWE-190 [2]

Description

`SafeMath` is a widely-used Solidity `math` library that is designed to support safe `math` operations by preventing common overflow or underflow issues when working with `uint256` operands. While it indeed blocks common overflow or underflow issues, the lack of `float` support in `Solidity` may introduce another subtle, but troublesome issue: precision loss. In this section, we examine one possible precision loss source that stems from the different orders when both multiplication (`mul`) and division (`div`) are involved.

In particular, we use the `InflowStreamImpl::_notifyInflowStreamAmount()` as an example. This routine is used to update stream state when tokens are added.

```
    function _notifyInflowStreamAmount(
        uint256 inflowStreamId_,
        uint256 amount_
    ) internal {
        updateInflowStreams(address(0));

        InflowStream storage inflowStream = inflowStreams[inflowStreamId_];

        if (inflowStream.endTime <= block.timestamp) {
            revert ErrInflowStreamHasEnded();
        }

        inflowStream.amount += amount_;
        inflowStreamsTotals[inflowStream.token] += amount_;
```

```
862         inflowStream.rate +=
863             amount_ /
864             (inflowStream.endTime -
865                 _max(inflowStream.startTime, block.timestamp));

867         inflowStream.lastUpdateTime = block.timestamp;
868     }
```

Listing 3.8: InflowStreamImpl::_notifyInflowStreamAmount()

We notice the calculation of the resulting `inflowStream.rate` (lines 862-865) is calculated by directly dividing the added amount by the time duration without multiplying a scaling factor of `_RAY`.[1]. With that, it may introduce unnecessary precision loss. Also, the `FinancingMarketAdapter::createWithdrawRequest()` function can be improved by calculating a flooring conversion from the given share to the underlying asset amount (line 903). Similarly, another `PoolImpl::maxDeposit()` function can be improved by calculating a ceiling conversion from user balance to underlying asset (line 466).

Recommendation Revise the above calculations to better mitigate possible precision loss.

Status This issue has been fixed by this commit: `db5a5a8`.

[1]Note the scaling factor is later multiplied when calculating the inflow stream rate, i.e., `Shared::inflowStreamRate()`.

3.9 Improper Weighted Rate Accounting During Write-Off

- ID: PVE-009
- Severity: High
- Likelihood: High
- Impact: High
- Target: `PerpHelpers`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

The financing module in `Atomica` supports undercollateralized lending, which means bad debt - financings where the borrower stops repaying - might occur. When it occurs, any remaining unpaid obligation is written off. Our analysis on the write-off mechanism shows an issue that does not properly update the weighted rate accounting for interest accumulation.

To elaborate, we show below the code snippet of the related routine, i.e., `writeOff()`. In this routine, there is a global state `weightedRate` that is used to keep track of the weighted average interest rate across all active financings. As mentioned earlier in Section 3.3, there are two types of financing for interest calculation, i.e., `INTEREST_ACCRUAL` and `FIXED_REPAYMENT`. The first type accrues borrow interest depends on actual duration while the second type has a fixed upfront repayment amount at financing disbursement and the interest does not change regardless of when borrower repays. In other words, only the first type will actually update `weightedRate` when a financing request is written off. Apparently, current implementation does the write-off regardless of the financing type (line 623).

```
589     function writeOff(
590         uint256 obligationId_
591     ) external override nonReentrant obligationExists(obligationId_) {
592         Obligation storage obligation = obligationsById[obligationId_];
593
594         if (obligation.market != msg.sender) {
595             revert ErrNotAllowed();
596         }
597
598         uint256 remainingPrincipal = obligation.borrowedPrincipal -
599             obligation.repaidPrincipal;
600         uint256 remainingInterest = interestObligations(obligationId_);
601
602         // Snapshot global accrued interest before modifying weightedRate
603         if (lastUpdateTs > 0) {
604             accruedInterest += weightedRate * (block.timestamp - lastUpdateTs);
605         }
606         lastUpdateTs = block.timestamp;
607
608         // Remove this obligation's interest from global tracking
609         if (obligation.rate == 0) {
610             // Fixed repayment: remove pre-scaled interest from global
```

```
            accruedInterest -= obligation.accruedInterest;
            fixedInterestObligations -= remainingInterest;
        } else {
            // Accrual interest: remove this obligation's stored + elapsed contribution
            accruedInterest -=
                obligation.accruedInterest +
                remainingPrincipal *
                obligation.rate *
                (block.timestamp - obligation.lastUpdateTs);
        }

        // Remove from weighted rate
        weightedRate -= remainingPrincipal * obligation.rate;

        // Remove from total borrowed principal (LP loss)
        totalBorrowedPrincipal -= remainingPrincipal;

        // Zero out the obligation
        obligation.accruedInterest = 0;
        obligation.borrowedPrincipal = obligation.repaidPrincipal; //
            principalObligations() -> 0
        obligation.rate = 0;
        obligation.lastUpdateTs = block.timestamp;

        emit LogObligationWrittenOff(obligationId_, remainingPrincipal,
            remainingInterest);
    }
```

Listing 3.9: `FinancingMarketAdapter::writeOff()`

Recommendation Revise the above `writeOff()` routine to properly update the `weightedRate` state.

Status This issue has been fixed by this commit: `36697ba`.

3.10 Revisited fixedInterestObligations Accounting in payInterest()

- ID: PVE-010
- Severity: Medium
- Likelihood: High
- Impact: Medium
- Target: `FinancingMarketAdapter`
- Category: Business Logic [7]
- CWE subcategory: CWE-770 [4]

Description

As mentioned earlier, `Atomica` allows for asset borrowing from a so-called adapter that bridges lending pools and outflows for liquidity management. In the process of examining the logic to pay interest, we notice an issue that does not properly update `fixedInterestObligations`, a global state to keep track of the total unpaid fixed interest across all fixed financing obligations.

To elaborate, we show below the implementation of the related routine, i.e., `payInterest()`, that processes interest payment for an obligation position. For a repayment of a `FIXED_REPAYMENT` obligation, we shall update `fixedInterestObligations` accordingly. It comes to our attention that the amount to deduct the interest from `fixedInterestObligations` should be `amount`, not current `netInterest` (line 540). The reason is that the operator fee is deducted from the paid interest and the operator fee is the only difference between `amount` and `fixedInterestObligations`.

```
489     function payInterest(
490         uint256 obligationId_,
491         uint256 amount_
492     ) external override nonReentrant obligationExists(obligationId_) returns (uint256) {
493         uint256 interestOwed = interestObligations(obligationId_);
494
495         if (amount_ == 0 || amount_ > interestOwed) {
496             revert ErrInvalidAmount();
497         }
498
499         Obligation storage obligation = obligationsById[obligationId_];
500
501         if (obligation.market != msg.sender) {
502             revert ErrNotAllowed();
503         }
504
505         accruedInterest =
506             accruedInterest +
507             weightedRate *
508             (block.timestamp - lastUpdateTs) -
509             amount_ *
510             _ONE;
511         lastUpdateTs = block.timestamp;
```

```

        if (obligation.rate == 0) {
            // Fixed repayment: store remaining interest in _ONE-scaled format
            obligation.accruedInterest = (interestOwed - amount_) * _ONE;
        } else {
            // Interest accrual: mirror global _ONE-scaled pattern to preserve precision
            obligation.accruedInterest =
                obligation.accruedInterest +
                (obligation.borrowedPrincipal - obligation.repaidPrincipal) *
                obligation.rate *
                (block.timestamp - obligation.lastUpdateTs) -
                amount_ *
                _ONE;
        }
        obligation.lastUpdateTs = block.timestamp;

        // Calculate and deduct operator fee immediately
        uint256 operatorFee = IPool(pool).curatorFee();
        uint256 fee = 0;
        uint256 netInterest = amount_;

        if (operatorFee > 0) {
            fee = (amount_ * operatorFee) / _ONE;
            fees += fee;
            netInterest = amount_ - fee;
        }

        if (obligation.rate == 0) {
            fixedInterestObligations -= netInterest;
        }

        paidInterest += netInterest;
        obligation.paidInterest += netInterest;

        CallHelper.safeTransferFrom(asset, msg.sender, address(this), amount_);

        emit LogInterestPaid(obligationId_, amount_);

        return amount_;
    }
```

Listing 3.10: `FinancingMarketAdapter::payInterest()`

Recommendation Revise the above `payInterest()` routine to properly update the `fixedInterestObligations` state for fixed financing obligations.

Status This issue has been fixed by this commit: `7cc5f90`.

3.11 Accommodation of Non-ERC20-Compliant Tokens

- ID: PVE-011
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `RepaymentLib`
- Category: Coding Practices [6]
- CWE subcategory: CWE-1126 [1]

Description

Though there is a standardized ERC-20 specification, many token contracts may not strictly follow the specification or have additional functionalities beyond the specification. In this section, we examine the `approve()` routine and analyze possible idiosyncrasies from current widely-used token contracts.

In particular, we use the popular stablecoin, i.e., `USDT`, as our example. We show the related code snippet below. On its entry of `approve()`, there is a requirement, i.e., `require(!((_value != 0) && (allowed[msg.sender][_spender] != 0)))`. This specific requirement essentially indicates the need of reducing the allowance to 0 first (by calling `approve(_spender, 0)`) if it is not, and then calling a second one to set the proper allowance. This requirement is in place to mitigate the known `approve()/transferFrom()` race condition (https://github.com/ethereum/EIPs/issues/20#issuecomment-263524729).

```
    /**
    * @dev Approve the passed address to spend the specified amount of tokens on behalf
        of msg.sender.
    * @param _spender The address which will spend the funds.
    * @param _value The amount of tokens to be spent.
    */
    function approve(address _spender, uint _value) public onlyPayloadSize(2 * 32) {

        // To change the approve amount you first have to reduce the addresses`
        //  allowance to zero by calling `approve(_spender, 0)` if it is not
        //  already 0 to mitigate the race condition described here:
        //  https://github.com/ethereum/EIPs/issues/20#issuecomment-263524729
        require(!((_value != 0) && (allowed[msg.sender][_spender] != 0)));

        allowed[msg.sender][_spender] = _value;
        Approval(msg.sender, _spender, _value);
    }
```

Listing 3.11: USDT Token **Contract**

Because of that, a normal call to `approve()` is suggested to use the safe version, i.e., `safeApprove()` , In essence, it is a wrapper around ERC20 operations that may either throw on failure or return false without reverts. Moreover, the safe version also supports tokens that return no value (and instead revert or throw on failure). Note that non-reverting calls are assumed to be successful. Similarly, there is a safe version of `transfer()` as well, i.e., `safeTransfer()`.

```
    /**
     * @dev Deprecated. This function has issues similar to the ones found in
     * {IERC20-approve}, and its usage is discouraged.
     *
     * Whenever possible, use {safeIncreaseAllowance} and
     * {safeDecreaseAllowance} instead.
     */
    function safeApprove(
        IERC20 token,
        address spender,
        uint256 value
    ) internal {
        // safeApprove should only be called when setting an initial allowance,
        // or when resetting it to zero. To increase and decrease it, use
        // 'safeIncreaseAllowance' and 'safeDecreaseAllowance'
        require(
            (value == 0) || (token.allowance(address(this), spender) == 0),
            "SafeERC20: approve from non-zero to non-zero allowance"
        );
        _callOptionalReturn(token, abi.encodeWithSelector(token.approve.selector,
            spender, value));
    }
```

Listing 3.12: `SafeERC20::safeApprove()`

In current implementation, if we examine the `RepaymentLib::payInterestFull()` routine, it is designed to pay all outstanding interest across all financing tranches. To accommodate the specific idiosyncrasy, it has correctly used `safeApprove()` (line 50). However, to ensure its proper operation, it also needs to reset the allowance to zero and then set the intended allowance.

```
36  function payInterestFull(
37      IFinancingMarket.Tranche[] storage tranches_,
38      address asset_,
39      uint256 financingId_
40  ) external returns (uint256 paidInterest_) {
41      for (uint256 i = 0; i < tranches_.length; i++) {
42          IFinancingMarket.Tranche storage tranche = tranches_[i];
43          IFinancingMarketAdapter adapter = IFinancingMarketAdapter(
44                  tranche.adapter
45              );
46
47          uint256 obligation = adapter.interestObligations(tranche.obligationId);
48          if (obligation == 0) continue;
49
50          CallHelper.safeApprove(asset_, tranche.adapter, obligation);
51
52          uint256 repaid = adapter.payInterest(tranche.obligationId, obligation);
53          paidInterest_ += repaid;
54
55          emit IFinancingMarket.LogInterestPaid(
56              financingId_,
57              tranche.pool,
```

```
                repaid,
                i
            );
        }
    }
```

Listing 3.13: `RepaymentLib::payInterestFull()`

Note a number of other routines share the same issue, including `payInterestProrata()`, `repayPrincipal()`, `payOtherObligations()`, and `liquidateTranches()`.

Recommendation Accommodate the above-mentioned idiosyncrasy about ERC20-related `approve()`.

Status This issue has been fixed by this commit: `2114240`.

3.12 Trust Issue of Admin Keys

- ID: PVE-012
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [5]
- CWE subcategory: CWE-287 [3]

Description

In the `Atomica` protocol, there are special administrative accounts, i.e., `operators` and `approvers`. These accounts play a critical role in governing and regulating the protocol-wide operations (e.g., configure various settings and execute privileged operations). They also have the privilege to control or govern the flow of assets within the protocol contracts. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
    function setHooksConfig(
        HooksConfig memory hooksConfig_
    )
        external
        onlyOperator
        delayedExecution(
            keccak256(abi.encodePacked(msg.sig)),
            keccak256(abi.encode(msg.sig, hooksConfig_)),
            executionDelay
        )
    {
        _validateHooksConfig(hooksConfig_);
        hooksConfig = hooksConfig_;
```

```
        emit LogHooksConfigSet(hooksConfig_);
    }

    /// @inheritdoc IFinancingMarket
    function setHooksEnabled(bool enabled_) external override {
        _checkOperator();
        ...
    }
    ...
    function setAccessTokensListId(uint256 accessTokensListId_) external {
        _checkOperator();
        ...
    }

    /// @inheritdoc IFinancingMarket
    function setAllowedAdaptersListId(
        uint256 allowedAdaptersListId_
    )
        external
        override
        onlyOperator
        delayedExecution(
            keccak256(abi.encodePacked(msg.sig)),
            keccak256(abi.encodePacked(msg.sig, allowedAdaptersListId_)),
            executionDelay
        )
    { ...}

    /// @inheritdoc IFinancingMarket
    function setTranchesLimit(uint256 tranchesLimit_) external {
        _checkOperator();
        ...
    }

    /// @inheritdoc IFinancingMarket
    function setLiquidationGracePeriod(uint256 liquidationGracePeriod_) external
        override {
        _checkOperator();
        ...
    }

    /// @inheritdoc IFinancingMarket
    function setLiquidatorsListId(
        uint256 liquidatorsListId_
    ) external override {
        _checkOperator();
        ...
    }

    /// @inheritdoc IFinancingMarket
    function setApproveCooldown(
        uint256 approveCooldown_
```

```
    ) external override {
        _checkOperator();
        ...
    }
    ...
    function setApproveExpiry(uint256 approveExpiry_) external override {
        _checkOperator();
        ...
    }

    /// @inheritdoc IFinancingMarket
    function setApprover(
        address approver_
    )
        external
        onlyOperator
        delayedExecution(
            keccak256(abi.encodePacked(msg.sig)),
            keccak256(abi.encodePacked(msg.sig, approver_)),
            executionDelay
        )
    { ...}

    /// @inheritdoc IFinancingMarket
    function setExposureCap(uint256 max_) external override {
        _checkOperator();
        ...
    }
```

Listing 3.14: Example Privileged Operations in `FinancingMarket`

We understand the need of the privileged functions for proper contract operations, but at the same time the extra power to these privileged accounts may also be a counter-party risk to the contract users. Therefore, we list this concern as an issue here from the audit perspective and highly recommend making these privileges explicit or raising necessary awareness among protocol users.

In the meantime, the audited protocol makes use of the proxy contracts to allow for future upgrades. The upgrade is a privileged operation and the management of the related admin key also falls in this trust issue.

Recommendation Promptly transfer the privileged account to the intended `DAO`-like governance contract. All changes to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status The issue has been mitigated with the planned use of multisig wallets to hold all admin roles, not `EOA (externally owned accounts)`.

4 Conclusion

In this audit, we have analyzed the design and implementation of the core `Atomica` protocol, an algorithmic protocol that aims to provide financial flexibility and reduce risk for protocol users by offering a range of protection cover options and underwriting capital for liquidity providers. This audit covers the core financing markets to enable peer-to-peer financing on top of pools, with intent-based matching, access control (via `ERC721` tokens), and an extensible hooks system. Each pool is in essence an `ERC4626` vault with integrated access control, inflow streams, phased lifecycle, and outflow processors. The current code base is clearly organized and those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that `Solidity`-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-190: Integer Overflow or Wraparound. https://cwe.mitre.org/data/definitions/190.html.

[3] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[4] MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. https://cwe.mitre.org/data/definitions/770.html.

[5] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[6] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[7] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[8] MITRE. CWE CATEGORY: Numeric Errors. https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[11] PeckShield. PeckShield Inc. https://www.peckshield.com.

PeckShield